

SOP: Bringing Up a NEMA 23 Joint

Arctos Robotic Arm — MKS_57D over CAN

Applies to Joint X

September 2026

Caution: This procedure has never been completed on real NEMA 23 hardware

No NEMA 23 joint on this arm has ever communicated over CAN. The MKS_57D driver installed on Joint X turned out to be the **RS485 variant**, which has no CAN transceiver, so every step below that involves the CAN protocol is extrapolated from the NEMA 17 / Servo42D work rather than confirmed on a 57D.

Each step is tagged **[VERIFIED]** or **[UNVERIFIED]**:

- **[VERIFIED]** — confirmed on this arm's hardware, and expected to carry over directly (host-side or physical steps).
- **[UNVERIFIED]** — assumed by analogy with the Servo42D. Plausible, but nobody has seen it work on a 57D. Confirm each of these as you go and update this document with what you actually observe.

Procedure only — for protocol details, encoder maths and troubleshooting, see `arctos_motor_guide.tex`.

Joint	X (base/waist rotation)
CAN ID	0x01 (per config; confirm by scan)
Gear ratio	13.5:1
Motor	NEMA 23, 1.8 N·m, 2.8 A, 6.35 mm shaft
Driver	MKS_57D

Caution: A NEMA 23 is substantially more dangerous than a NEMA 17

1.8 N·m through 13.5:1 is roughly **24 N·m at the joint** — about 75× the NEMA 17 joints' output torque, and more than enough to injure a hand or destroy a printed part before you can react. Keep clear of the mechanism, keep the emergency stop in a second terminal, and keep a hand on the power switch for every motion step.

Step 0 — Confirm the board is the CAN variant **[VERIFIED]**

Do this first. It is the failure that has already cost this project weeks.

1. Read the part number and silkscreen on the MKS_57D board.
2. Confirm it is the **CAN** variant, not RS485. The two look similar and both present a differential pair.

Caution: RS485 boards will never work, at any bitrate or address

RS485 carries MKS's own async serial protocol, not CAN framing. A CAN controller and an RS485 transceiver cannot talk over the same wires. If this board is RS485 you must either swap it for the CAN variant — which keeps this entire procedure and every script valid — or treat the joint as a separate RS485 subsystem with its own adapter and its own driver software, which is out of scope here.

Pass: Board is confirmed CAN variant.

If it fails: Stop. No amount of software work will make an RS485 board appear on the bus.

Step 1 — Pre-power electrical checks [VERIFIED]

1. Everything powered off, measure CAN_H to CAN_L: expect $\approx 60\ \Omega$.
2. Confirm only CAN_H, CAN_L, GND run between the CANable and the controller.
3. **Check the supply.** A NEMA 23 at 2.8 A draws far more than the NEMA 17 joints. Confirm the supply is rated for it, and that wiring to the driver is sized appropriately.
4. Confirm the joint is mechanically clear and you know its travel limits.

Pass: $\approx 60\ \Omega$, correct wiring, adequate supply.

Step 2 — Configure the controller panel [UNVERIFIED]

The 57D's menu layout has not been confirmed to match the 42D's. Set and **save**:

CAN ID	0x01
CAN Rate	500
CAN Response (CANRSP)	ENABLE
CAN CRC	SUM-8
Working current	$\leq 2.8\ \text{A}$ (the NEMA 23's rating)

Power-cycle after saving.

Caution: Set working current deliberately

A 42D in this project was found storing 3000 mA against a 1.2 A motor. On a 2.8 A NEMA 23 the consequences of an over-set current are correspondingly larger.

Pass: Settings persist across a power cycle.

If it fails: If the menu differs from the above, record what it actually offers and update this document — that is exactly the unverified part.

Step 3 — Bring up the CAN interface [VERIFIED]

Identical to the NEMA 17 procedure; this is host-side and motor-independent.

```
lsusb # expect ID 16d0:117e CANable2
ls -l /dev/serial/by-id/
sudo pkill slcand
sudo ip link delete can0 2>/dev/null
sudo slcand -o -c -s6 /dev/serial/by-id/<your-canable> can0
sudo ip link set can0 txqueuelen 1000
sudo ip link set up can0
ip -br link show can0
```

Pass: can0 UP

Step 4 — Confirm the real CAN ID [UNVERIFIED]

The scan itself is verified; what is unverified is whether a 57D answers the 0x31 query the same way a 42D does.

```
python3 src/arctos_can_control/arctos_can_control/can_id_scan.py \
--channel can0 --ids 1-16
```

Read-only — never enables coils, never commands motion.

Pass: Exactly one responder, with a valid checksum.

If it fails: Total silence with the board powered and wiring confirmed is the RS485 signature. Go back to Step 0 and check the part number before doing anything else. Note that the slcan adapter does not populate kernel bus-error counters, so you cannot use “zero errors” as corroboration here.

Step 5 — Read-only health check [UNVERIFIED]

```
python3 src/arctos_can_control/arctos_can_control/joint_reliability_test.py \
--channel can0 --can-id 0x01 --gear-ratio 13.5 --samples 200
```

Query	Opcode	Expected (by analogy)
Encoder	0x31	stable; 0 jitter, 0 drift
Velocity	0x32	zero
Error	0x39	small
Enable state	0x3A	01
Shaft protection	0x3E	00

Caution: Confirm the encoder resolution on the 57D

This procedure assumes 16,384 counts/revolution and 3200 microstep pulses/rev, as on the 42D. **Verify this before trusting any angle maths:** command a known number of pulses and check the encoder delta matches. If the 57D differs, every conversion in this SOP changes.

Pass: 100% response, 0 checksum failures, 0 jitter at rest.

Step 6 — Check the mechanism by hand [VERIFIED]

1. Disable coils: `cansend can0 001#F300F4`. (*Checksum: $0xF3+0x00+0x01 = 0xF4$.*)
 2. Confirm 0x3A now reads 00.
 3. Turn the joint output by hand, gently, both directions, watching the encoder.
-

Caution: Do not drop coils on a loaded base joint

Joint X carries the entire arm above it. Removing holding torque may let the whole assembly swing. Only do this with the arm supported or removed.

Pass: Encoder tracks your hand both ways; motion feels smooth.

If it fails: Gritty resistance means the gear mesh needs plastic-safe grease. No encoder change while the output turns means the motor is not coupled to it.

Step 7 — First commanded motion [UNVERIFIED]

Keep the emergency stop in a second terminal before sending anything:

```
cansend can0 001#F7F8
```

(*Checksum: $0xF7+0x01 = 0xF8$.*)

```
python3 src/arctos_can_control/arctos_can_control/joint_unstick_move_test.py \  
  --can-id 0x01 --gear-ratio 13.5 \  
  --step-degrees 0.25 --num-steps 1 --max-unstick 0 \  
  --band-lo -1 --band-hi 1 --speed 25
```

Caution: Start smaller than you would on a NEMA 17

At 13.5:1 the reduction is far lower than the NEMA 17 joints' 48:1 and 67.82:1, so the same commanded joint angle turns the motor far less — and the joint moves much faster for a given motor speed. A 0.25° joint step here is only 3.4° of motor rotation. Verify direction and magnitude on this first step before increasing anything.

Pass: ≈100% of commanded, with FD=[1, 2].

If it fails: Do not retry into a stall. Return to Step 6.

Step 8 — Stepped travel test [UNVERIFIED]

```
python3 src/arctos_can_control/arctos_can_control/joint_unstick_move_test.py \
    --can-id 0x01 --gear-ratio 13.5 \
    --step-degrees 2.0 --num-steps 5 --max-unstick 0 \
    --band-lo -15 --band-hi 15 --speed 25

# reverse, same step count
python3 src/arctos_can_control/arctos_can_control/joint_unstick_move_test.py \
    --can-id 0x01 --gear-ratio 13.5 \
    --step-degrees -2.0 --num-steps 5 --max-unstick 0 \
    --band-lo -15 --band-hi 15 --speed 25
```

Pass: Every step 99–101% with FD 02; round trip returns to start.

Step 9 — Establish this motor’s speed limit [UNVERIFIED]

The NEMA 17 figures do not transfer. The NEMA 23 joints run a Servo57D, where the NEMA 17 joints run a Servo42D — different winding inductance and a different torque-speed curve, so this motor’s usable speed ceiling must be measured, not assumed. The measured per-joint table in `arctos_motor_guide.tex` is for Servo42D joints; do not read Joint B’s numbers across to a 57D.

With the joint unloaded and clear, run short trials at increasing **speed** and record actual rpm against commanded. The usable limit is the highest speed where actual still tracks commanded to within about 1%; beyond that the motor silently loses steps.

Record the result here once measured:

Linear range (tracks commanded)	_____ rpm
Roll-off begins	_____ rpm
Saturation ceiling	_____ rpm

Table 1: To be filled in from measurement. For reference, the NEMA 17 joints track linearly to 225 rpm and saturate near 310 rpm.

Step 10 — Acceptance

Check	Criterion
Board variant	confirmed CAN, not RS485
CAN ID	confirmed by scan
Comms	100% response, 0 checksum failures
Encoder at rest	0 jitter, 0 drift
Encoder scale	confirmed for the 57D, not assumed
Hand check	smooth, tracks both directions
Per-step accuracy	99–101% of commanded
Status frames	FD 02 on every step
Round trip	returns to start
Speed limit	measured and recorded

Step 11 — Shutdown [VERIFIED]

```
# stop any running script first (Ctrl+C)
cansend can0 001#F300F4      # disable coils, only if safe to do so
sudo ip link set can0 down
sudo pkill slcand
```

Update this document

Every [UNVERIFIED] tag above is a gap. As each step is confirmed on real NEMA 23 hardware, change its tag to [VERIFIED] and record what was actually observed — in particular the panel menu layout, the encoder resolution, and the measured speed limits.